



Univerzitet u Nišu
Elektronski Fakultet



Replikacija PostgreSQL baze podataka korišćenjem Leader-Follower arhitekture

Mentor:
doc. dr Aleksandar Stanimirović

Student:
Filip Stamenković 2027

Niš, 2026.

Sadržaj

1. Uvod	3
2. Replikacija podataka u distribuiranim bazama podataka	4
2.1 Pojam i značaj replikacije	4
2.2 Modeli replikacije	4
2.3 Sinhrona i asinhrona replikacija	5
2.4 Prednosti i ograničenja replikacije	6
3. Leader-Follower arhitektura	7
3.1 Osnovni principi rada	7
3.2 Tok operacija upisa i čitanja	7
3.3 Prednosti Leader-Follower arhitekture	8
3.4 Ograničenja i izazovi	9
3.5 Failover mehanizmi	9
3.6 Primena Leader-Follower arhitekture u realnim sistemima	10
4. PostgreSQL Streaming Replication	11
4.1 Pregled PostgreSQL mehanizma replikacije	11
4.2 Write-Ahead Log (WAL)	11
4.3 WAL Sender i WAL Receiver procesi	13
4.4 Replication Slots	14
4.5 Hot Standby režim	14
4.6 Sinhrona i asinhrona replikacija u PostgreSQL-u	15
5. Praktična implementacija Leader-Follower arhitekture u PostgreSQL-u	16
5.1 Opis eksperimentalnog okruženja	16
5.2 Konfiguracija primarnog servera	16
5.3 Konfiguracija replike	17
5.4 Demonstracija rada replikacije	17
5.5 Analiza statusa replikacije	19
5.6 Otkaz replika servera i oporavak	19
5.7 Otkaz primarnog servera i promocija replika servera	20
5.8 Diskusija rezultata	21
6. Zaključak	22
7. Literatura	23

1. Uvod

Savremeni informacijski sistemi obrađuju sve veće količine podataka i istovremeno moraju da obezbede visoku dostupnost, pouzdanost i dobre performanse. Sa porastom broja korisnika i zahteva za neprekidnim radom sistema, tradicionalni pristup skladištenju podataka na jednom serveru često postaje nedovoljan. U takvim uslovima primena distribuiranih sistema baza podataka i različitih mehanizama replikacije predstavlja jedno od ključnih rešenja za povećanje dostupnosti i otpornosti sistema na otkaze.

Replikacija podataka podrazumeva održavanje više kopija iste baze podataka na različitim čvorovima sistema. Njena osnovna svrha je povećanje pouzdanosti, obezbeđivanje rezervnih kopija podataka i raspodela opterećenja između više servera. Korišćenjem replikacije moguće je smanjiti vreme nedostupnosti sistema u slučaju otkaza pojedinačnih komponenti, kao i unaprediti performanse čitanja podataka korišćenjem replika.

Jedan od najčešće korišćenih modela replikacije jeste Leader-Follower arhitektura. U ovom modelu jedan čvor ima ulogu glavnog servera (Leader) i zadužen je za obradu svih operacija upisa, dok jedan ili više pomoćnih čvorova (Follower) primaju promene sa glavnog servera i održavaju njegove kopije. Ovakav pristup omogućava jednostavnu implementaciju replikacije, efikasno skaliranje operacija čitanja i povećanje dostupnosti sistema.

PostgreSQL predstavlja jedan od najpopularnijih otvorenih sistema za upravljanje relacionim bazama podataka i pruža ugrađenu podršku za različite mehanizme replikacije. Kroz Streaming Replication mehanizam PostgreSQL omogućava kontinuirano prenošenje promena sa glavnog servera na replike korišćenjem WAL (Write-Ahead Log) zapisa. Na taj način moguće je realizovati Leader-Follower arhitekturu uz relativno jednostavnu konfiguraciju i visoku pouzdanost rada.

2. Replikacija podataka u distribuiranim bazama podataka

2.1 Pojam i značaj replikacije

Replikacija podataka predstavlja proces održavanja više kopija istih podataka na različitim čvorovima distribuiranog sistema. Ove kopije, poznate kao replike, sinhronizuju se kako bi obezbedile doslednost podataka i omogućile njihov pristup sa više lokacija. Replikacija je jedan od najvažnijih mehanizama u savremenim sistemima baza podataka jer doprinosi povećanju dostupnosti, pouzdanosti i skalabilnosti sistema [1].

U sistemima sa velikim brojem korisnika i visokim zahtevima za dostupnošću, oslanjanje na jedan server može predstavljati značajan rizik. Otkaz hardvera, problemi sa mrežom ili preopterećenje sistema mogu dovesti do prekida rada aplikacije. Korišćenjem replikacije moguće je smanjiti posledice ovakvih situacija jer se podaci nalaze na više čvorova koji mogu preuzeti obradu zahteva u slučaju problema sa glavnim serverom.

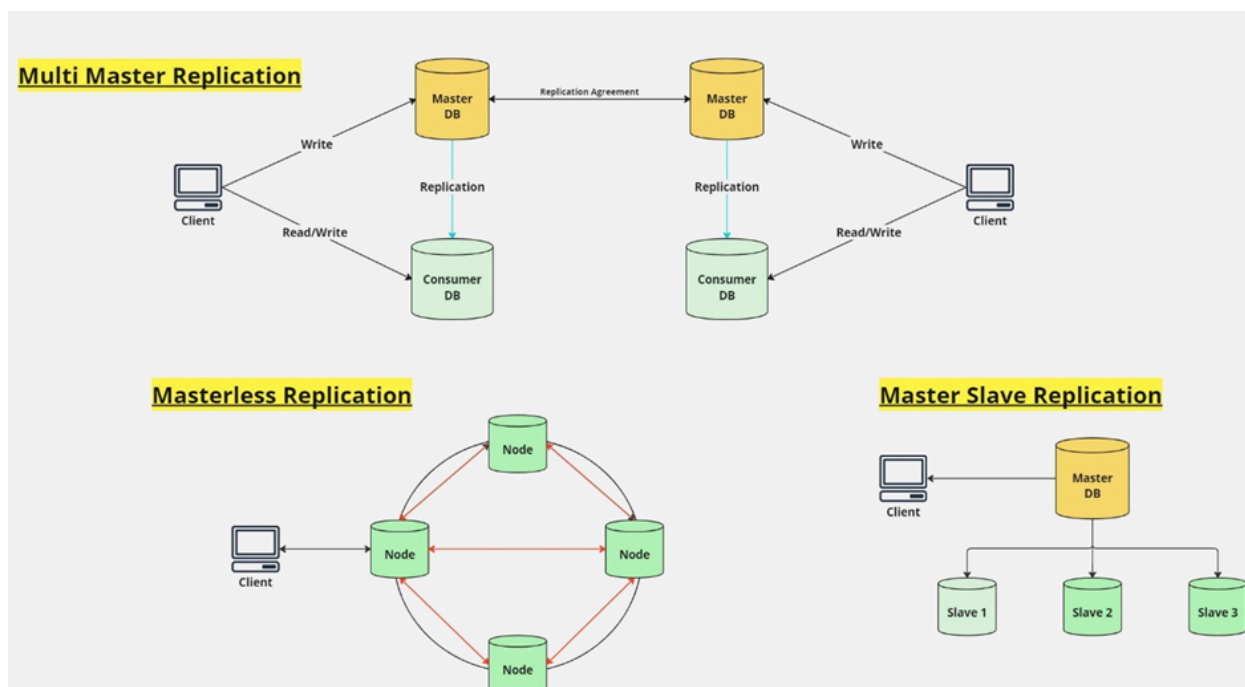
Pored povećanja dostupnosti, replikacija omogućava raspodelu opterećenja između više servera. Operacije čitanja mogu se izvršavati na replikama, dok glavni server obrađuje operacije upisa. Na taj način smanjuje se opterećenje centralnog čvora i poboljšavaju performanse sistema, naročito u aplikacijama koje generišu veliki broj zahteva za čitanje podataka.

2.2 Modeli replikacije

U distribuiranim bazama podataka postoji više modela organizacije replika, pri čemu svaki model poseduje određene prednosti i ograničenja [1].

Najčešće korišćen model jeste Leader-Follower arhitektura. U ovom modelu jedan čvor ima ulogu glavnog servera (Leader) i zadužen je za obradu svih operacija upisa. Ostali čvorovi funkcionišu kao replike (Followers) koje primaju promene od glavnog servera i održavaju njegove kopije. Ovaj model je relativno jednostavan za implementaciju i široko je zastupljen u modernim DBMS sistemima.

Multi-Leader model omogućava postojanje više čvorova koji mogu prihvatati operacije upisa. Promene izvršene na jednom čvoru propagiraju se ka ostalim čvorovima sistema. Ovakav pristup povećava dostupnost i omogućava geografski distribuirane upise, ali uvodi dodatne izazove vezane za rešavanje konflikata između istovremenih izmena podataka.



Slika 1: Modeli replikacije - Multi Master(Multi Leader), Masterless (Peer to Peer), Master Slave (Leader Follower).

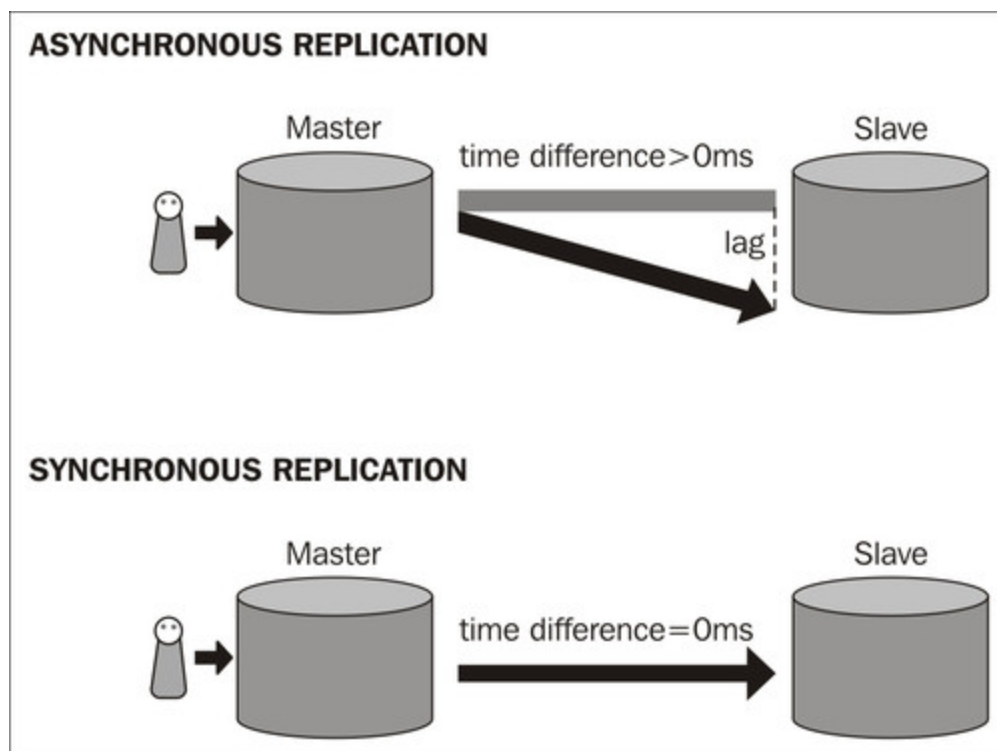
Kod Peer-to-Peer modela ne postoji centralni čvor. Svi serveri imaju ravnopravnu ulogu i mogu prihvatati zahteve za čitanje i upis podataka. Ovakva arhitektura pruža visok stepen otpornosti na otkaze, ali je značajno složenija za implementaciju i održavanje zbog problema vezanih za sinhronizaciju i konzistentnost podataka.

2.3 Sinhrona i asinhrona replikacija

Jedna od najvažnijih karakteristika sistema replikacije odnosi se na način prenosa promena između čvorova. U zavisnosti od toga razlikuju se sinhrona i asinhrona replikacija.

Kod sinhronne replikacije, operacija upisa smatra se uspešno završenom tek nakon što je promena potvrđena na svim ili unapred definisanom broju replika. Ovakav pristup obezbeđuje visok nivo konzistentnosti podataka jer su sve replike u svakom trenutku usklađene sa glavnim serverom. Međutim, dodatna komunikacija između čvorova može povećati vreme izvršavanja operacija i negativno uticati na performanse sistema.

Kod asinhronne replikacije, glavni server potvrđuje uspešan upis odmah nakon lokalnog izvršavanja operacije, dok se promene naknadno prenose na replike. Prednost ovog pristupa jeste manja latencija i bolje performanse, ali postoji mogućnost da replike privremeno zaostaju za glavnim serverom. Ovo kašnjenje poznato je kao replication lag i predstavlja jedno od najvažnijih ograničenja asinhronih sistema.



Slika 2: Razlika između sinhronne i asinhronne replikacije.

Izbor između sinhronne i asinhronne replikacije zavisi od zahteva konkretnog sistema. Sistemi kojima je prioritet konzistentnost podataka češće koriste sinhronu replikaciju, dok sistemi kojima su važnije performanse i dostupnost često primenjuju asinhroni pristup [1].

2.4 Prednosti i ograničenja replikacije

Primena replikacije donosi brojne prednosti u radu sistema baza podataka. Jedna od najvažnijih jeste povećanje dostupnosti podataka. U slučaju otkaza pojedinačnog servera, replika može preuzeti njegovu funkciju i omogućiti nastavak rada sistema sa minimalnim prekidom.

Replikacija takođe omogućava skaliranje operacija čitanja raspodelom zahteva između više čvorova. Ovakav pristup posebno je koristan kod sistema sa velikim brojem korisnika gde većinu opterećenja predstavljaju operacije čitanja podataka.

Sa druge strane, replikacija uvodi dodatnu složenost u administraciju sistema. Potrebno je obezbediti pravilnu konfiguraciju, nadzor rada replika i rešavanje eventualnih problema sa sinhronizacijom podataka. Kod asinhronne replikacije može doći do pojave replication lag-a, što može uzrokovati privremene razlike između podataka na glavnom serveru i replikama [1].

Uprkos navedenim izazovima, replikacija predstavlja osnovni mehanizam za izgradnju pouzdanih i skalabilnih distribuiranih sistema baza podataka. Zbog toga je njena primena postala standard u velikom broju savremenih DBMS rešenja.

3. Leader-Follower arhitektura

3.1 Osnovni principi rada

Leader-Follower arhitektura predstavlja jedan od najrasprostranjenijih modela replikacije u savremenim sistemima baza podataka. Osnovna ideja ovog pristupa zasniva se na postojanju jednog glavnog čvora, označenog kao Leader, i jednog ili više pomoćnih čvorova, označenih kao Followers. Leader je odgovoran za obradu svih operacija koje menjaju stanje baze podataka, dok Followers održavaju kopije podataka i sinhronizuju se sa glavnim čvorom [1].

Kada korisnik izvrši operaciju upisa, kao što su INSERT, UPDATE ili DELETE, zahtev se prosleđuje Leader čvoru. Nakon uspešnog izvršavanja operacije, promena se prenosi ka Followers čvorovima putem mehanizma replikacije. Na taj način replike održavaju ažurne kopije podataka i mogu biti korišćene za obradu zahteva za čitanje.

Ovakav pristup omogućava jasno razdvajanje operacija čitanja i upisa. Centralizovanjem svih operacija upisa na jednom mestu pojednostavljuje se održavanje konzistentnosti podataka, dok se istovremeno postiže mogućnost raspodele opterećenja čitanja na više servera.

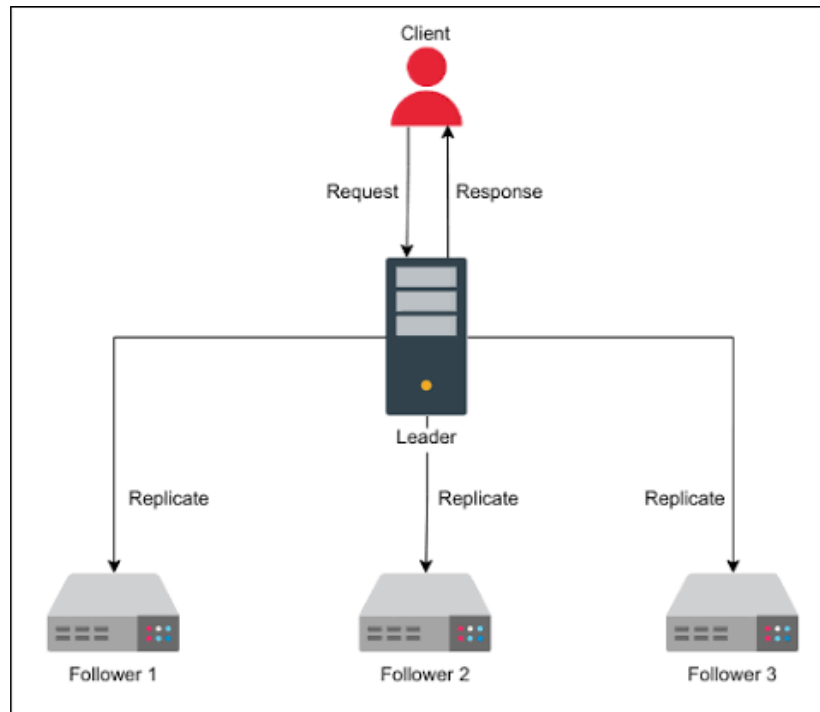
3.2 Tok operacija upisa i čitanja

U Leader-Follower arhitekturi svi zahtevi za izmenu podataka obrađuju se isključivo na Leader čvoru. Nakon izvršavanja operacije, promena se evidentira u logovima sistema i prenosi na Followers čvorove. Replike zatim primenjuju iste promene kako bi njihovo stanje ostalo usklađeno sa glavnim serverom [1].

Proces izvršavanja operacije upisa može se prikazati sledećim koracima:

1. Klijent šalje zahtev za upis podataka.
2. Leader izvršava operaciju nad bazom podataka.
3. Promena se zapisuje u log sistema.
4. Promena se prenosi ka Followers čvorovima.
5. Replike primenjuju primljene izmene.

Za razliku od operacija upisa, zahtevi za čitanje mogu biti usmereni i na Leader i na Followers čvorove. U sistemima sa velikim brojem korisnika često se koristi raspodela operacija čitanja na replike kako bi se smanjilo opterećenje glavnog servera i poboljšale performanse sistema.



Slika 3: Tok operacija kod Leader-Follower arhitekture.

3.3 Prednosti Leader-Follower arhitekture

Jedna od najvažnijih prednosti Leader-Follower modela jeste jednostavnost implementacije i administracije. Pošto se svi upisi izvršavaju na jednom mestu, lakše je održavati konzistentnost podataka i pratiti tok promena kroz sistem.

Dodatna prednost ogleda se u mogućnosti skaliranja operacija čitanja. Kako broj korisnika raste, moguće je dodavati nove replike koje će preuzimati deo opterećenja vezanog za čitanje podataka. Na taj način sistem može opslužiti veći broj zahteva bez značajnog povećanja opterećenja glavnog servera.

Leader-Follower arhitektura doprinosi i povećanju dostupnosti sistema. U slučaju otkaza jednog od Followers čvorova sistem nastavlja da funkcioniše bez značajnih posledica. Replike se takođe mogu koristiti za kreiranje rezervnih kopija podataka i implementaciju Disaster Recovery (oporavak od katastrofa) strategija.

Još jedna značajna prednost jeste mogućnost geografske distribucije replika. Organizacije često postavljaju replike u različitim regionima kako bi korisnici pristupali podacima sa lokacije koja im je fizički bliža, čime se smanjuje mrežna latencija.

3.4 Ograničenja i izazovi

Iako poseduje brojne prednosti, Leader-Follower arhitektura ima i određena ograničenja. Najznačajnije ograničenje predstavlja činjenica da postoji samo jedan čvor koji obrađuje operacije upisa. Sa povećanjem broja zahteva upravo Leader postaje potencijalno usko grlo sistema.

Dodatni izazov predstavlja kašnjenje u replikaciji podataka. Kod asinhronne replikacije Follower čvorovi mogu privremeno zaostajati za glavnim serverom, što može dovesti do situacija u kojima korisnik na replici vidi zastarele podatke [1]. Ova pojava poznata je kao replication lag.

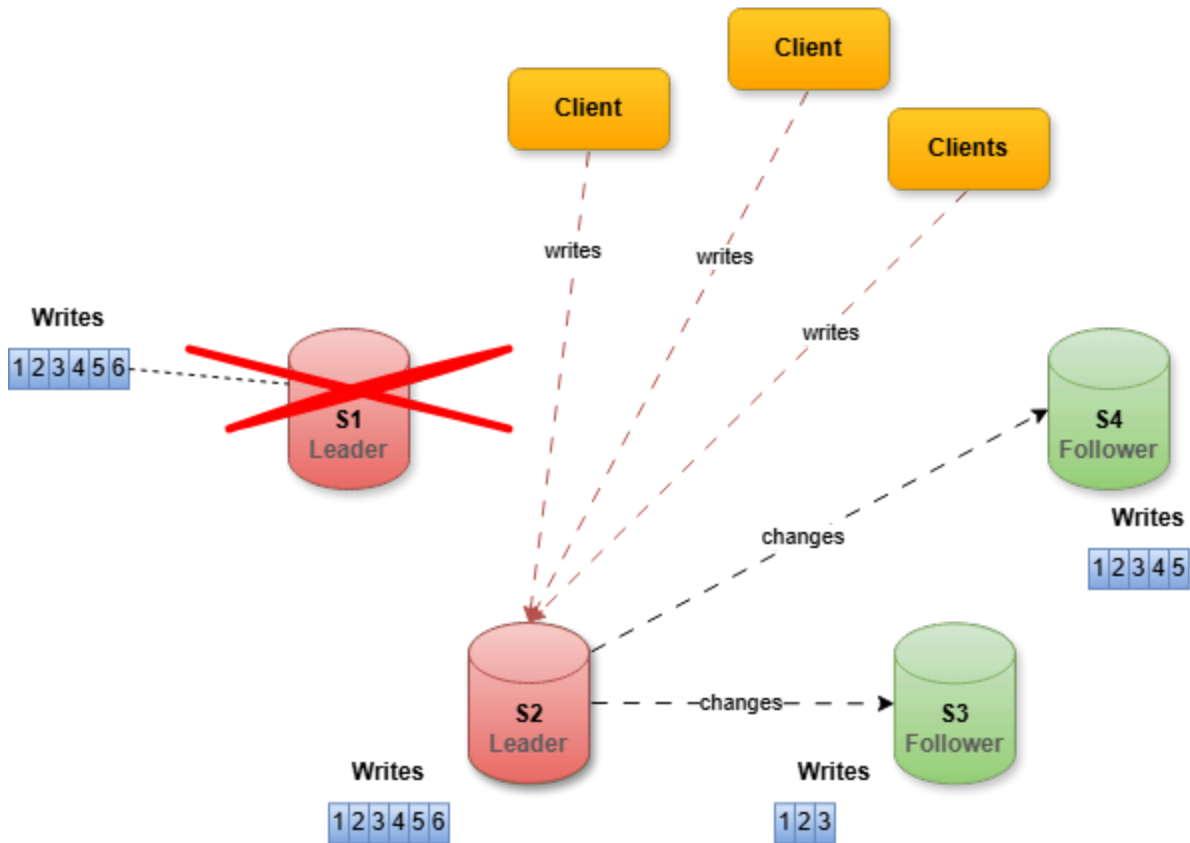
Takođe, neophodno je voditi računa o pravilnoj konfiguraciji i nadzoru sistema. Problemi sa mrežom, prekidi komunikacije ili neuspešna sinhronizacija mogu dovesti do odstupanja između replika i glavnog servera.

Zbog navedenih ograničenja Leader-Follower arhitektura najčešće se koristi u sistemima gde dominiraju operacije čitanja, dok broj operacija upisa ostaje na nivou koji jedan glavni server može efikasno da obradi.

3.5 Failover mehanizmi

Failover predstavlja proces automatskog ili manuelnog prebacivanja rada sistema na rezervni čvor u slučaju otkaza glavnog servera. U Leader-Follower arhitekturi ovaj mehanizam ima ključnu ulogu u obezbeđivanju dostupnosti sistema.

U slučaju otkaza Leader čvora, jedan od Followers čvorova može biti promovisan u novog Leader-a. Nakon promocije, novi glavni server preuzima obradu operacija upisa, dok se ostale replike povezuju na njega i nastavljaju proces replikacije [1][2].



Slika 4: Failover Leader čvora i promocija Followera u novi Leader čvor.

Failover može biti realizovan ručno od strane administratora ili automatski korišćenjem specijalizovanih alata za nadzor i upravljanje replikacijom. Automatizovani pristupi omogućavaju kraće vreme oporavka sistema i smanjenje perioda nedostupnosti usluga.

U PostgreSQL okruženju failover se često implementira korišćenjem alata kao što su Patroni, repmgr ili Pgpool-II, koji omogućavaju nadzor rada čvorova i automatsku promociju replika u slučaju otkaza primarnog servera.

3.6 Primena Leader-Follower arhitekture u realnim sistemima

Leader-Follower arhitektura široko je zastupljena u savremenim informacionim sistemima zbog svoje jednostavnosti, pouzdanosti i mogućnosti skaliranja operacija čitanja. Mnoge organizacije koriste ovaj model kako bi obezbedile visoku dostupnost podataka i efikasnu raspodelu opterećenja između servera.

Jedna od najčešćih oblasti primene jesu sistemi elektronske trgovine (e-commerce). U ovakvim sistemima veliki broj korisnika istovremeno gleda proizvode, pretražuje kataloge i pristupa informacijama o narudžbinama. Pošto operacije čitanja značajno nadmašuju broj operacija upisa, zahtevi za čitanje mogu se usmeriti ka replikama, dok se operacije kreiranja i izmene narudžbina izvršavaju na primarnom serveru.

Društvene mreže takođe predstavljaju značajan primer primene Leader-Follower arhitekture. Korisnici konstantno pristupaju objavama, komentarima i profilima drugih korisnika, što generiše veliki broj operacija čitanja. Korišćenjem replika moguće je distribuirati opterećenje i omogućiti brži pristup podacima većem broju korisnika.

U bankarskim i finansijskim sistemima replikacija se koristi prvenstveno radi povećanja dostupnosti i pouzdanosti podataka. Iako su zahtevi za konzistentnošću podataka veoma visoki, replike se često koriste za izveštavanje, analitiku i kreiranje rezervnih kopija bez dodatnog opterećenja primarnog servera.

Cloud servisi i SaaS platforme predstavljaju još jednu oblast u kojoj je Leader-Follower arhitektura veoma zastupljena. Veliki broj korisnika pristupa istim podacima sa različitih geografskih lokacija, pa se replike često raspoređuju u više regiona kako bi se smanjila mrežna latencija i povećala dostupnost sistema. Na taj način korisnici ostvaruju brži pristup podacima, dok se istovremeno obezbeđuje otpornost na otkaze pojedinačnih data centara.

4. PostgreSQL Streaming Replication

4.1 Pregled PostgreSQL mehanizma replikacije

PostgreSQL predstavlja jedan od najpopularnijih sistema za upravljanje relacionim bazama podataka otvorenog koda. Kako bi podržao zahteve savremenih informacionih sistema za visokom dostupnošću i skalabilnošću, PostgreSQL obezbeđuje više mehanizama replikacije podataka. Najčešće korišćen pristup jeste Streaming Replication, koji omogućava kontinuirani prenos promena sa primarnog servera na jednu ili više replika [2].

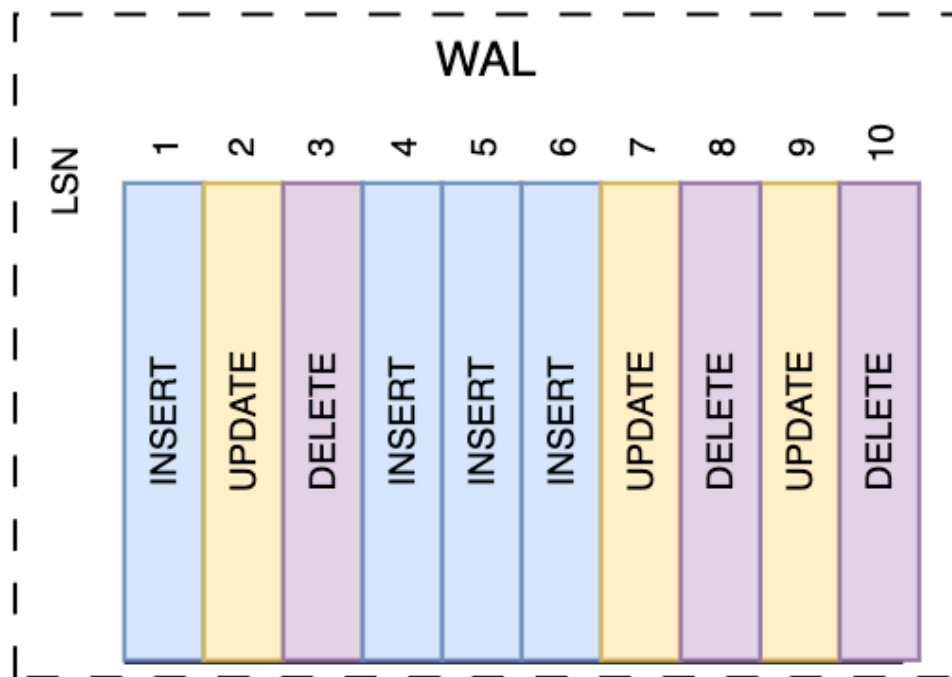
Streaming Replication se zasniva na prenosu WAL (Write-Ahead Log) zapisa između čvorova sistema. Svaka promena izvršena nad bazom podataka prvo se evidentira u WAL zapisima, nakon čega se promene mogu proslediti replikama radi održavanja njihove usklađenosti sa primarnim serverom [2]. Ovakav pristup omogućava efikasnu sinhronizaciju podataka uz minimalan uticaj na performanse sistema.

U kontekstu Leader-Follower arhitekture, primarni PostgreSQL server ima ulogu Leader čvora, dok replike funkcionišu kao Followers. Primarni server prihvata operacije upisa i generiše WAL zapise, dok replike kontinuirano preuzimaju i primenjuju te zapise kako bi održavale identičnu kopiju baze podataka.

4.2 Write-Ahead Log (WAL)

Write-Ahead Log predstavlja jedan od osnovnih mehanizama PostgreSQL sistema za obezbeđivanje integriteta i oporavka podataka. Princip rada WAL-a zasniva se na tome da se

svaka promena prvo zapisuje u log datoteke pre nego što bude trajno upisana u samu bazu podataka.



Slika 5: Struktura WAL zapisa.

Ovakav pristup omogućava oporavak sistema nakon neočekivanih prekida rada, kao što su otkaz hardvera ili nestanak električne energije. Tokom procesa oporavka PostgreSQL može koristiti WAL zapise kako bi ponovo primenio sve potvrđene transakcije koje još nisu bile trajno zapisane u datoteke baze podataka.

Pored funkcije oporavka sistema, WAL predstavlja osnovu za implementaciju Streaming Replication mehanizma [2]. Umesto prenosa kompletnih tabela ili baza podataka, PostgreSQL prenosi samo WAL zapise koji sadrže informacije o izvršenim promenama. Na taj način značajno se smanjuje količina podataka koja se prenosi između čvorova.

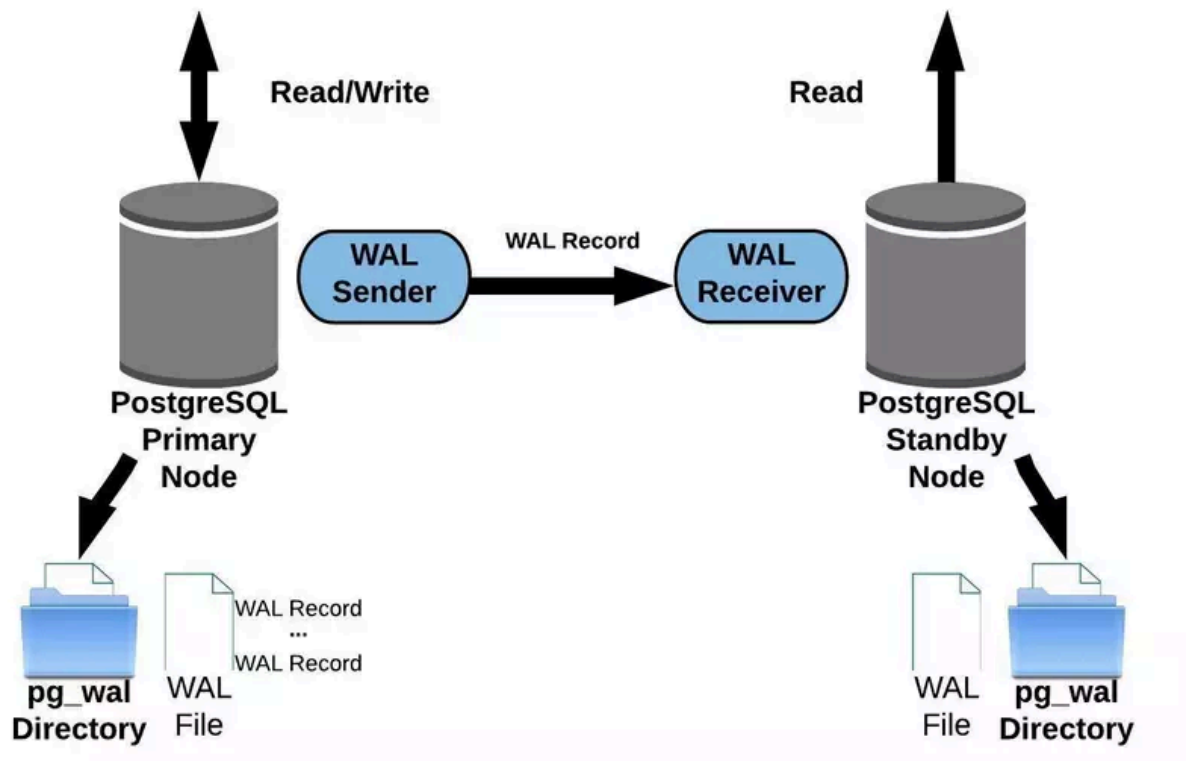
Proces korišćenja WAL mehanizma može se predstaviti sledećim koracima:

1. Korisnik izvršava operaciju nad bazom podataka.
2. PostgreSQL generiše odgovarajuće WAL zapise.
3. WAL zapisi se trajno evidentiraju na disku.
4. Promene se primenjuju nad podacima baze.
5. WAL zapisi se šalju replikama radi sinhronizacije.

4.3 WAL Sender i WAL Receiver procesi

Streaming Replication koristi specijalizovane procese za prenos WAL zapisa između čvorova sistema. Na primarnom serveru aktivan je WAL Sender proces, dok se na repliki izvršava WAL Receiver proces [2][7].

WAL Sender proces zadužen je za slanje WAL zapisa ka povezanim replikama. Kada replika uspostavi vezu sa primarnim serverom, WAL Sender započinje prenos svih potrebnih log zapisa kako bi replika mogla da dostigne trenutno stanje sistema.



Slika 6: Tok operacija WAL Sender-a i WAL Receiver-a.

Sa druge strane, WAL Receiver proces prima WAL zapise sa primarnog servera i skladišti ih lokalno na replici. Nakon prijema, posebni PostgreSQL procesi primenjuju primljene promene na lokalnu kopiju baze podataka.

Komunikacija između WAL Sender i WAL Receiver procesa odvija se kontinuirano, što omogućava gotovo trenutno prenošenje promena između čvorova. Vreme potrebno da replika primeni promene zavisi od mrežnih uslova, opterećenja sistema i konfiguracije replikacije.

4.4 Replication Slots

Replication Slots predstavljaju mehanizam koji PostgreSQL koristi kako bi osigurao da replike ne propuste WAL zapise neophodne za sinhronizaciju. Bez korišćenja ovog mehanizma postoji mogućnost da primarni server ukloni stare WAL datoteke pre nego što ih replika preuzme, što može dovesti do prekida procesa replikacije [3].

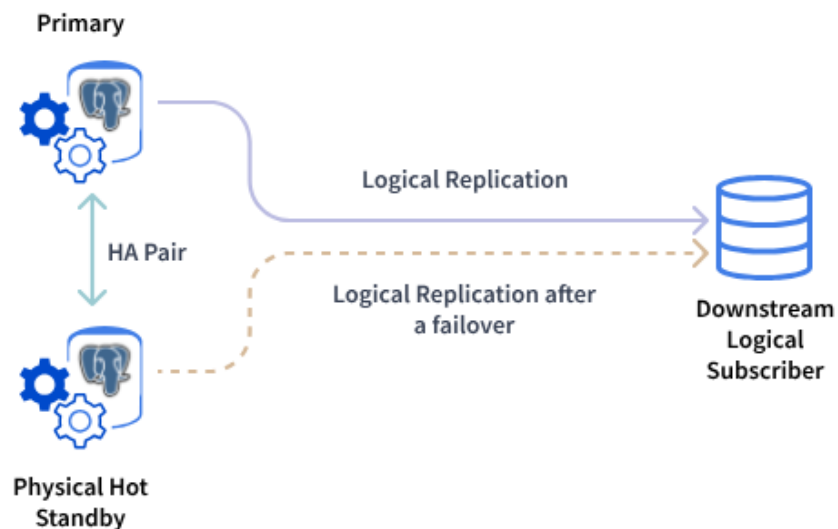
Kada replika koristi Replication Slot, primarni server prati njen napredak i zadržava potrebne WAL zapise dok replika ne potvrdi njihov uspešan prijem. Na taj način obezbeđuje se pouzdana sinhronizacija čak i u situacijama kada replika privremeno izgubi mrežnu konekciju ili bude nedostupna.

Iako povećavaju pouzdanost sistema, Replication Slots zahtevaju pažljivo upravljanje. Ukoliko replika duži vremenski period ne preuzima podatke, WAL datoteke mogu zauzeti značajan prostor na disku primarnog servera [3].

4.5 Hot Standby režim

Hot Standby predstavlja režim rada PostgreSQL replike koji omogućava izvršavanje operacija čitanja nad replikom tokom procesa replikacije. Bez ovog režima replika bi služila isključivo za održavanje rezervne kopije podataka [2].

Korišćenjem Hot Standby funkcionalnosti moguće je usmeriti deo korisničkih zahteva za čitanje ka replikama i time smanjiti opterećenje primarnog servera. Ovakav pristup posebno je koristan kod sistema u kojima dominiraju operacije čitanja podataka.



Slika 7: Tok operacija u Hot Standby režimu.

Važno je naglasiti da replike u Hot Standby režimu ne prihvataju operacije koje menjaju podatke. Pokušaj izvršavanja INSERT, UPDATE ili DELETE naredbi rezultuje greškom jer se na repliki održava režim samo za čitanje (read-only).

Zahvaljujući ovoj funkcionalnosti PostgreSQL omogućava efikasno skaliranje čitanja podataka uz zadržavanje jednostavne Leader-Follower arhitekture i visokog nivoa konzistentnosti između čvorova sistema.

4.6 Sinhrona i asinhrona replikacija u PostgreSQL-u

PostgreSQL podržava i sinhroni i asinhroni način rada replikacije, čime omogućava prilagođavanje sistema različitim zahtevima u pogledu performansi i konzistentnosti podataka [1]. Izbor odgovarajućeg pristupa zavisi od potreba konkretnog informacionog sistema i prioriteta između brzine izvršavanja transakcija i nivoa zaštite podataka.

Podrazumevana konfiguracija PostgreSQL Streaming Replication mehanizma zasniva se na asinhronoj replikaciji. U ovom režimu primarni server potvrđuje uspešno izvršavanje transakcije odmah nakon lokalnog upisa podataka, bez čekanja da replike prime i primene odgovarajuće WAL zapise. Prednost ovakvog pristupa ogleda se u maloj latenciji i visokim performansama sistema. Međutim, u slučaju iznenadnog otkaza primarnog servera postoji mogućnost da deo promena još nije prenet na replike, što može dovesti do gubitka najnovijih transakcija.

Za razliku od asinhronog pristupa, kod sinhrono replikacije primarni server čeka potvrdu od jedne ili više replika pre nego što transakciju označi kao uspešno izvršenu. Na taj način postiže se viši nivo konzistentnosti podataka i smanjuje mogućnost njihovog gubitka u slučaju otkaza primarnog servera. Cena ovakvog pristupa jeste povećanje vremena izvršavanja transakcija zbog dodatne komunikacije između čvorova.

U PostgreSQL sistemu sinhrona replikacija može se konfigurisati korišćenjem parametra [3]:

```
synchronous_commit = on
```

Ovaj parametar određuje da li primarni server treba da čeka potvrdu o prijemu WAL zapisa pre završetka transakcije.

Pored toga, parametar [3]:

```
synchronous_standby_names = 'replica1'
```

omogućava definisanje replika koje učestvuju u sinhronoj replikaciji. Moguće je navesti jednu ili više replika čije potvrde su neophodne za uspešno izvršavanje transakcije.

Asinhrona replikacija najčešće se koristi u sistemima gde su performanse i odziv aplikacije važniji od apsolutne konzistentnosti podataka. Sa druge strane, sinhrona replikacija primenjuje

se u sistemima sa strogim zahtevima za zaštitu podataka, kao što su bankarski, finansijski i pojedini industrijski sistemi.

Izbor između sinhronog i asinhronog modela predstavlja kompromis između performansi i pouzdanosti. PostgreSQL pruža podršku za oba pristupa, omogućavajući administratorima da odaberu konfiguraciju koja najbolje odgovara potrebama konkretnog sistema.

5. Praktična implementacija Leader-Follower arhitekture u PostgreSQL-u

5.1 Opis eksperimentalnog okruženja

U cilju praktične demonstracije Leader-Follower arhitekture implementirano je okruženje zasnovano na PostgreSQL Streaming Replication mehanizmu. Celokupno okruženje realizovano je korišćenjem Docker Compose platforme, čime je omogućena jednostavna konfiguracija, pokretanje i reprodukcija eksperimenta.

Implementirano okruženje sastoji se od dva PostgreSQL servera:

- **postgres-primary** – primarni (Leader) server koji prihvata sve operacije upisa;
- **postgres-replica** – sekundarni (Follower) server koji održava kopiju podataka primarnog servera.

Oba servera povezana su u zajedničku Docker mrežu, dok svaki server poseduje sopstveni persistentni volume za čuvanje podataka. Na ovaj način replika predstavlja potpuno nezavisnu instancu baze podataka, a ne deljeni direktorijum podataka.

Za potrebe demonstracije korišćena je PostgreSQL verzija 18, dok je komunikacija između servera ostvarena putem ugrađenog Streaming Replication mehanizma.

5.2 Konfiguracija primarnog servera

Prvi korak implementacije podrazumevao je konfiguraciju primarnog PostgreSQL servera za rad u režimu replikacije.

Prilikom pokretanja servera definisani su osnovni parametri neophodni za Streaming Replication:

```
wal_level = replica  
max_wal_senders = 10
```



```
max_replication_slots = 10
wal_keep_size = 256MB
```

Parametar *wal_level* omogućava generisanje WAL zapisa potrebnih za replikaciju, dok *max_wal_senders* određuje maksimalan broj procesa koji mogu istovremeno slati WAL zapise replikama. Parametar *max_replication_slots* omogućava korišćenje replication slot-ova, dok *wal_keep_size* definiše minimalnu količinu WAL podataka koja će biti zadržana radi uspešne sinhronizacije replika [3].

Tokom inicijalizacije baze automatski je kreiran poseban korisnik namenjen procesu replikacije:

```
CREATE ROLE replicator
WITH LOGIN
REPLICATION
PASSWORD 'replicator123';
```

Pored toga, u datoteku *pg_hba.conf* dodato je pravilo kojim se dozvoljava uspostavljanje replication konekcije između primarnog servera i replike.

5.3 Konfiguracija replike

Za kreiranje replike korišćen je PostgreSQL alat **pg_basebackup**, koji omogućava pravljenje fizičke kopije postojećeg PostgreSQL klastera [4].

Prilikom pokretanja replike izvršava se skripta koja najpre proverava dostupnost primarnog servera. Nakon uspešnog povezivanja pokreće se komanda **pg_basebackup**, kojom se kopiraju svi podaci sa primarnog servera i automatski konfiguriše replika za rad u standby režimu.

Tokom ovog procesa kreiran je i fizički replication slot naziva **replica_slot**, čime je omogućeno da primarni server zadržava WAL zapise sve dok ih replika uspešno ne preuzme.

Nakon završetka inicijalizacije replika automatski prelazi u režim rada i započinje kontinuirano preuzimanje WAL zapisa sa primarnog servera.

5.4 Demonstracija rada replikacije

Nakon uspešnog uspostavljanja replikacije izvršena je provera ispravnosti sistema korišćenjem jednostavne test tabele.

Najpre je na primarnom serveru kreirana tabela:

```
CREATE TABLE users(
    id SERIAL PRIMARY KEY,
    name TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
```

Nakon toga izvršen je unos podataka:

```
INSERT INTO users(name)
VALUES ('Filip');
```

Na repliki je izvršena naredba:

```
SELECT * FROM users;
```

Rezultat je pokazao da je novi zapis uspešno prenet sa primarnog servera na repliku.

```
PS C:\Users\filip\Documents\GitHub\leader-follower-db-replication-postgresql> docker exec -it postgres-primary psql -U admin -d bank
psql (18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

bank=# sql/exp_1.sql
bank=# sql/exp_1.sql
bank=# \i sql/exp_1.sql
CREATE TABLE
INSERT 0 1
 id | name |      created_at
-----+-----+-----
  1 | Filip | 2026-08-10 20:56:19.790884
(1 row)

bank=#
```

```
d bank
Error response from daemon: No such container: postgres-secondary
PS C:\Users\filip\Documents\GitHub\leader-follower-db-replication-postgresql> docker exec -it postgres-replica psql -U admin -d bank
psql (18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

bank=# SELECT * FROM users;
 id | name |      created_at
-----+-----+-----
  1 | Filip | 2026-08-10 20:56:19.790884
(1 row)

bank=# ^C
bank=#
```

Slika 8: Rezultat uspešnog prenosa novog zapisa iz Leader baze u Follower bazu.

Pored unosa novih podataka, izvršene su i operacije ažuriranja i brisanja postojećih zapisa. Sve promene izvršene na primarnom serveru bile su gotovo trenutno vidljive na repliki, čime je potvrđen ispravan rad Streaming Replication mehanizma.

Radi dodatne provere izvršen je i unos većeg broja zapisa korišćenjem SQL funkcije **generate_series()**. Nakon završetka izvršavanja potvrđeno je da je replika uspešno preuzela sve izmene bez gubitka podataka.

```
bank=# \i sql/exp_4.sql
INSERT 0 10000
 count
-----
 10000
(1 row)

bank=#
```

```
bank=# SELECT COUNT(*) FROM users;
 count
-----
 10000
(1 row)

bank=#
```

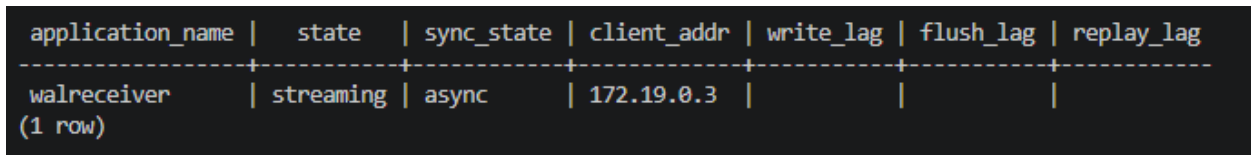
Slika 9: Rezultat unosa većeg broja zapisa i uspešan prenos istih u Follower bazu.

5.5 Analiza statusa replikacije

Status uspostavljene replikacije analiziran je korišćenjem PostgreSQL sistemskog pogleda **pg_stat_replication** [5].

Izvršavanjem odgovarajućeg SQL upita potvrđeno je da je replika uspešno povezana sa primarnim serverom i da se nalazi u stanju **streaming**, što označava aktivan prenos WAL zapisa.

Takođe je potvrđeno da sistem koristi asinhroni način replikacije (**sync_state = async**), što predstavlja podrazumevanu konfiguraciju PostgreSQL Streaming Replication mehanizma.



application_name	state	sync_state	client_addr	write_lag	flush_lag	replay_lag
walreceiver	streaming	async	172.19.0.3			

(1 row)

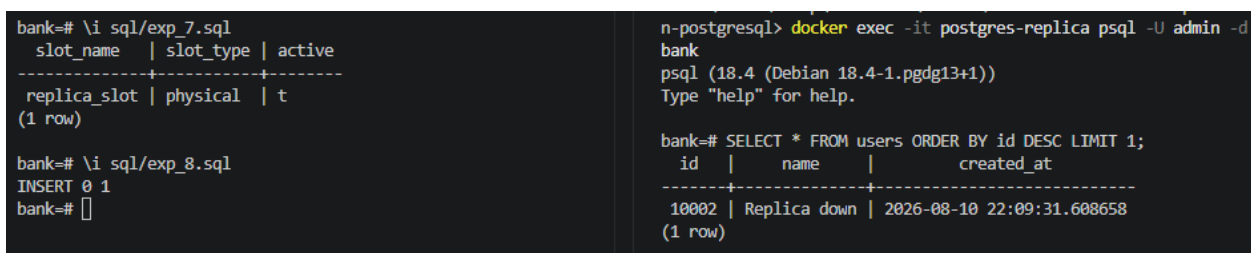
Slika 10: Status replikacije dobijen pg_stat_replication pogledom.

Pored toga, analizirani su i aktivni replication slot-ovi korišćenjem pogleda **pg_replication_slots**, pri čemu je potvrđeno da je kreirani fizički replication slot aktivan i pravilno povezan sa replikom.

5.6 Otkaz replika servera i oporavak

Kako bi se proverila otpornost sistema na otkaz sekundarnog servera, replika je privremeno zaustavljena korišćenjem Docker komande.

Tokom perioda nedostupnosti replike primarni server je nastavio normalno da prihvata nove transakcije i obrađuje zahteve korisnika.



slot_name	slot_type	active
replica_slot	physical	t

(1 row)

id	name	created_at
10002	Replica down	2026-08-10 22:09:31.608658

(1 row)

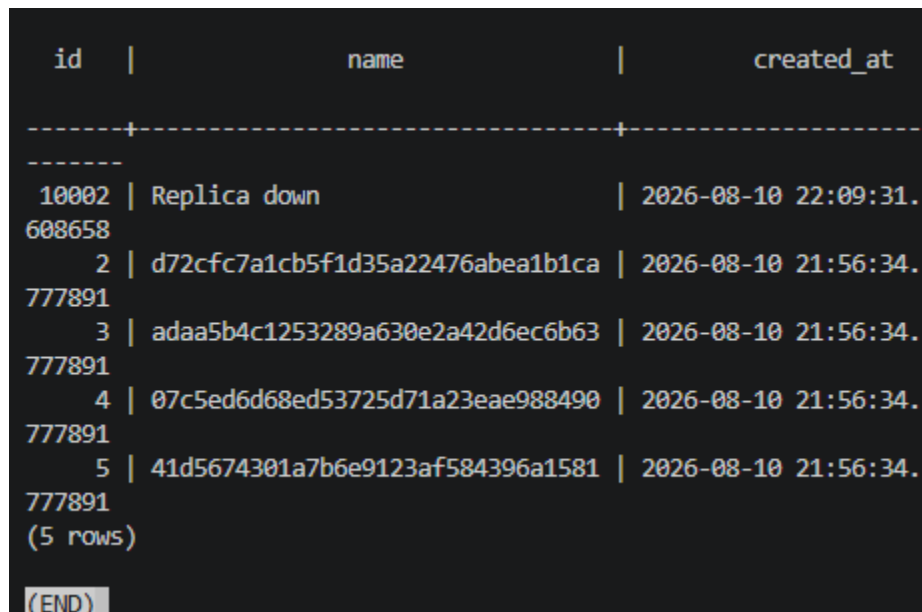
Slika 11: Rezultat uspešne replikacije nakon pada i ponovnog podizanja replike.

Nakon ponovnog pokretanja replike uspostavljena je nova konekcija sa primarnim serverom, nakon čega su svi propušteni WAL zapisi automatski preuzeti i primenjeni. Na ovaj način replika je uspešno dostigla trenutno stanje baze podataka bez potrebe za ručnom intervencijom administratora [2][3].

5.7 Otkaz primarnog servera i promocija replika servera

U narednom eksperimentu simuliran je otkaz primarnog servera njegovim zaustavljanjem.

Nakon prestanka rada primarnog servera potvrđeno je da replika i dalje omogućava izvršavanje operacija čitanja, ali nije moguće izvršavati operacije koje menjaju podatke, budući da se replika i dalje nalazi u recovery režimu.

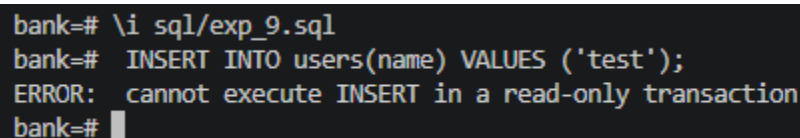


id	name	created_at
10002	Replica down	2026-08-10 22:09:31.
608658		
2	d72cfc7a1cb5f1d35a22476abea1b1ca	2026-08-10 21:56:34.
777891		
3	adaa5b4c1253289a630e2a42d6ec6b63	2026-08-10 21:56:34.
777891		
4	07c5ed6d68ed53725d71a23eae988490	2026-08-10 21:56:34.
777891		
5	41d5674301a7b6e9123af584396a1581	2026-08-10 21:56:34.
777891		

(5 rows)

(END)

Slika 12: Rezultat uspešne operacije čitanja sa replike nakon pada primarnog servera.



```
bank=# \i sql/exp_9.sql
bank=# INSERT INTO users(name) VALUES ('test');
ERROR: cannot execute INSERT in a read-only transaction
bank=#
```

Slika 13: Demonstracija da je replika nakon pada primarnog servera i dalje u read-only režimu rada.

Kako bi replika preuzela ulogu glavnog servera, izvršena je PostgreSQL funkcija [6]:

```
SELECT pg_promote();
```

Nakon uspešne promocije potvrđeno je da funkcija **pg_is_in_recovery()** vraća vrednost **false**, čime je potvrđeno da replika više ne radi u standby režimu.

```

bank=# SELECT pg_promote();
pg_promote
-----
t
(1 row)

bank=# SELECT pg_is_in_recovery();
pg_is_in_recovery
-----
f
(1 row)

bank=# \i sql/exp_10.sql
INSERT 0 1
bank=#

```

Slika 14: Rezultat uspješne promocije replike u primarni server (uspješan upis dokazuje da je replika sada primarni server).

Nakon promocije uspješno su izvršene nove INSERT operacije, čime je potvrđeno da je replika preuzela ulogu primarnog servera.

Ovaj eksperiment pokazuje da PostgreSQL Streaming Replication sam po sebi ne obezbeđuje automatski failover, već je za potpuno automatizovano prebacivanje na rezervni server potrebno koristiti dodatne alate kao što su Patroni ili repmgr [2].

5.8 Diskusija rezultata

Rezultati sprovedenih eksperimenata pokazali su da PostgreSQL Streaming Replication predstavlja pouzdan mehanizam za implementaciju Leader-Follower arhitekture. Sve izmene izvršene na primarnom serveru uspješno su prenete na repliku uz veoma malo kašnjenje, dok je korišćenje replike omogućilo izvršavanje operacija čitanja bez dodatnog opterećenja primarnog servera.

Eksperimenti su takođe pokazali da sistem uspješno podnosi privremeni otkaz replike, pri čemu se nakon njenog ponovnog pokretanja automatski uspostavlja sinhronizacija podataka. Sa druge strane, simulacija otkaza primarnog servera pokazala je da PostgreSQL Streaming Replication ne omogućava automatski failover, već zahteva ručnu promociju replike ili upotrebu dodatnih alata za visoku dostupnost.

Na osnovu sprovedene implementacije može se zaključiti da je Leader-Follower arhitektura pogodna za sisteme kod kojih dominiraju operacije čitanja i gde postoji potreba za povećanjem dostupnosti i pouzdanosti baze podataka, uz relativno jednostavnu konfiguraciju i održavanje.

6. Zaključak

Replikacija podataka predstavlja jedan od najvažnijih mehanizama za obezbeđivanje visoke dostupnosti, pouzdanosti i skalabilnosti savremenih sistema baza podataka. Sa razvojem distribuiranih informacionih sistema i sve većim zahtevima za neprekidnim radom aplikacija, primena odgovarajućih strategija replikacije postala je neizostavan deo projektovanja i implementacije baza podataka.

U ovom radu analizirana je Leader-Follower arhitektura kao jedan od najčešće korišćenih modela replikacije. Objasnjeni su osnovni principi njenog rada, način obrade operacija čitanja i upisa, kao i njene prednosti i ograničenja. Posebna pažnja posvećena je PostgreSQL sistemu za upravljanje bazama podataka i njegovom Streaming Replication mehanizmu, koji omogućava efikasnu implementaciju Leader-Follower arhitekture korišćenjem WAL zapisa.

U praktičnom delu rada implementirano je replikaciono okruženje zasnovano na PostgreSQL sistemu, pri čemu je demonstrisana uspešna sinhronizacija podataka između primarnog servera i replike. Izvršeni testovi pokazali su da se promene izvršene na primarnom serveru pouzdano prenose na repliku, kao i da replika može efikasno preuzeti obradu operacija čitanja, čime se smanjuje opterećenje primarnog servera i povećava ukupna skalabilnost sistema.

Rezultati sprovedenog eksperimenta potvrđuju da PostgreSQL Streaming Replication predstavlja pouzdano i praktično rešenje za implementaciju Leader-Follower arhitekture u sistemima kod kojih dominiraju operacije čitanja i gde postoji potreba za visokom dostupnošću podataka. Istovremeno, analiza je pokazala da ovaj pristup ima određena ograničenja, pre svega postojanje jednog čvora zaduženog za operacije upisa i mogućnost pojave kašnjenja u replikaciji kod asinhronog načina rada.

7. Literatura

- [1] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA, USA: O'Reilly Media, 2017. Dostupno: <https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063>
- [2] PostgreSQL Global Development Group, "Log-Shipping Standby Servers," *PostgreSQL 18 Documentation*, 2026. Dostupno: <https://www.postgresql.org/docs/18/warm-standby.html>
- [3] PostgreSQL Global Development Group, "Replication," *PostgreSQL 18 Documentation*, 2026. Dostupno: <https://www.postgresql.org/docs/18/runtime-config-replication.html>
- [4] PostgreSQL Global Development Group, "pg_basebackup," *PostgreSQL 18 Documentation*, 2026. Dostupno: <https://www.postgresql.org/docs/18/app-pgbasebackup.html>
- [5] PostgreSQL Global Development Group, "The Cumulative Statistics System," *PostgreSQL 18 Documentation*, 2026. Dostupno: <https://www.postgresql.org/docs/18/monitoring-stats.html>
- [6] PostgreSQL Global Development Group, "System Administration Functions," *PostgreSQL 18 Documentation*, 2026. Dostupno: <https://www.postgresql.org/docs/18/functions-admin.html>
- [7] PostgreSQL Global Development Group, "Streaming Replication Protocol," *PostgreSQL 18 Documentation*, 2026. Dostupno: <https://www.postgresql.org/docs/18/protocol-replication.html>